Perfect—this is one of the **most commonly asked JVM interview topics**. Let's walk through a **full Minor GC cycle step-by-step**, including **Eden → Survivor → Promotion to Old Gen**.

🧩 Heap Layout (Before GC)

```
Young Generation:
  Eden        S0 (From)      S1 (To)
 [ A B C D ] [   -   ]   [   -   ]

Old Generation:
 [   X Y   ]
```

- New objects are created in **Eden**
- Survivor spaces: **S0 (From)** and **S1 (To)**
- Assume:
  - A, B, C, D are newly created
  - X, Y are long-lived objects in Old Gen

🔄 Step 1: Eden Gets Full → Trigger Minor GC

👉 Minor GC starts when **Eden is full**

🎨 Step 2: Mark Live Objects (Roots Scan)

GC finds reachable objects from:

- Stack
- Registers
- Static variables

Assume:

- A, C are reachable ✅
- B, D are garbage ❌

📦 Step 3: Copy Live Objects → Survivor Space (S1)

👉 Young GC uses **copying algorithm**

```
Before:
Eden: [ A B C D ]
S0:   [ - ]
S1:   [ - ]

After:
Eden: [ - - - - ]   (cleared)
S1:   [ A C ]       (copied here)
```

- Only **live objects are copied**
- Garbage (B, D) is ignored → reclaimed instantly

🔢 Step 4: Assign Age

Each object gets an **age counter**

```
A (age=1), C (age=1)
```

🔁 Step 5: Swap Survivor Spaces

👉 S1 becomes **From**, S0 becomes **To**

```
S0 (To):   [ - ]
S1 (From): [ A C ]
```

🆕 Step 6: New Allocations Happen

Now new objects arrive:

```
Eden: [ E F G ]
S1:   [ A C ]
```

🔄 Step 7: Next Minor GC

Assume:

- A is still alive
- C is dead
- E, F alive
- G dead

📦 Copy Again

Live objects:

- From Eden → E, F
- From S1 → A

👉 Copy into S0:

```
S0 (To): [ A E F ]
```

🔢 Update Ages

```
A (age=2)
E (age=1)
F (age=1)
```

🚀 Step 8: Promotion to Old Generation

If:

- Object age ≥ threshold (e.g., 15)
  OR
- Survivor space is full

👉 Object is **promoted to Old Gen**

Example

If A reaches threshold:

```
Old Gen:
[ X Y A ]
```

⚠️ Important Edge Case: Survivor Overflow

If Survivor space can't hold all live objects:

👉 Some objects are **promoted early** to Old Gen

🧠 Full Flow Summary

1. Allocate in Eden
2. Eden full → Minor GC
3. Copy live → Survivor (To)
4. Increment age
5. Swap Survivor spaces
6. Repeat
7. Promote long-lived objects → Old Gen

🔥 Key Optimizations

1️⃣ Bump Pointer Allocation

- Eden allocation is just pointer increment → super fast

2️⃣ Copying = No Fragmentation

- Survivor spaces always compact

3️⃣ Most Objects Die Early

- So copying cost is minimal

⚠️ Important Interview Detail

Root Set Includes Old ⟶ Young References

If:

```
Old → Young reference exists
```

👉 Minor GC must consider it

Handled using:

- **Card table**
- **Remembered set**

🧠 Visualization (Mental Model)

Think of it like:

- Eden = **newborn zone**
- Survivor = **testing zone**
- Old Gen = **retirement home**

Only “survivors” get promoted 😄

🔥 Interview One-Liner

> “Minor GC copies live objects from Eden and Survivor spaces into another Survivor space, increments their age, and promotes long-lived objects to the Old Generation.”

🚀 If You Want to Go Deeper

Next-level questions I can walk you through:

- How **card tables / remembered sets** actually work internally
- What is **tenuring threshold tuning**
- How **G1 handles young generation differently**
- What causes **promotion failure → Full GC**

Just tell me 👍